

KITT INDUSTRIES

Python-Based Intelligent Car Manufacturing Management System (ICMMS)

Technical Report - Assessment WRIT1

Student Name: [Your Name]

Student ID: [Your Student ID]

Module: [Module Name]

Date: [Submission Date]

Word count (main body): approximately 2000 words

Contents

1. Introduction.....	2
2. Problem Summary.....	3
3. Technical Document Annex.....	3
3.1 Input-Process-Output (IPO) Analysis.....	3
3.2 Data Representation Plan.....	4
3.3 Function Planning (Modularisation Map).....	4
3.4 Algorithmic Logic Discussion.....	5
4. Python Implementation.....	6
5. Evidence of Program Run.....	6
6. Testing, Debugging and Documentation.....	7
6.1 Debugging.....	8
7. Limitations.....	8
8. Future Improvements.....	8
9. Reflection.....	9
10. Conclusion.....	9
Appendix A - Full Console Log of Program Run.....	11
Appendix B - Python Source Code (icmms.py).....	14

1. Introduction

The Intelligent Car Manufacturing Management System (ICMMS) is a menu-driven Python application designed for KITT Industries, a manufacturer of intelligent vehicles such as autonomous, AI-enabled and electric cars. The company previously relied on manual records and spreadsheets, which caused incorrect price tracking, poor visibility of models and slow retrieval of car details. This report documents the complete development of the system. Section 2 summarises the problem, while Section 3 presents the technical annex including the IPO analysis, data representation plan, modularisation map and algorithmic logic. Section 4 describes the implementation and coding standards, Section 5 provides evidence of program execution, and Section 6 presents testing and debugging. Sections 7 to 10 cover limitations, future improvements, reflection and conclusions. The report demonstrates how a modular, validated application improves accuracy and efficiency in managing the intelligent car inventory.

2. Problem Summary

KITT Industries manages a growing inventory of intelligent vehicles, yet its operations depend on manual records and spreadsheets. This outdated approach causes incorrect price entries, difficulty organising car records, poor visibility of models, delays in identifying affordable or premium cars and slow retrieval of car details. With limited staff and increasing demand, records are sometimes entered incorrectly, sorting by price or ID is time-consuming and manual searching is inefficient, causing delays in decision-making and confusion during customer inquiries. To overcome these issues, a menu-driven Python application is required that

allows staff to add, display, update, delete, search and sort car records, while validating all inputs and saving records to a file so that no data is lost.

3. Technical Document Annex

3.1 Input-Process-Output (IPO) Analysis

Table 1 summarises the data the program receives, how it is processed and the expected output in each case.

Table 1: Input-Process-Output (IPO) Analysis

Input	Process	Output
Car ID, name, category, price	Validate every field, then store the record	“Car added successfully”
Menu selection (1-9)	Route the choice to the matching function	Selected operation runs
Car ID	Search the list for a matching ID	Car details or “Car not found”
Sort option (price or ID)	Sort records using the selected key	Sorted car table
Car ID and new values	Update the fields of the matched record	“Car record updated successfully”
Car ID	Remove the matched record from the list	“Car record deleted successfully”
Inventory data	Compute totals, average, cheapest and most expensive	Inventory summary report
Save and exit	Write all records to the JSON file	File updated and clean exit

3.2 Data Representation Plan

All data in the ICMMS uses two simple Python structures: lists and dictionaries. Each car is a dictionary with four fields: id (a string such as C101), name (a string such as AutoDrive X), category (a string such as Autonomous) and price (a floating-point number). Dictionaries map field names to values, making each record readable and self-documenting. All records are held in a single list, which preserves insertion order and allows iteration, searching, sorting and modification. The inventory is persisted to

a JSON text file, `car_inventory.json`. JSON is a lightweight text-based format that can be edited in any text editor, satisfying the requirement for storage using a simple text file. On startup `load_inventory()` rebuilds the list from the file, and after every change `save_inventory()` writes the list back, preserving data types automatically. This keeps the design simple, scalable and aligned with the requirement to use simple data structures.

3.3 Function Planning (Modularisation Map)

The program is divided into small, single-purpose functions so that every task is independent, reusable and easy to test. Table 2 lists each function, its purpose and how it supports the modular design.

Table 2: Function Planning (Modularisation Map)

Function	Purpose	Support for Modular Design
<code>load_inventory()</code>	Reads records from the JSON file	Separates file handling from menu logic
<code>save_inventory(cars)</code>	Writes all records to the JSON file	Encapsulates persistence
<code>validate_car_id(cars, id)</code>	Checks ID format and uniqueness	Reusable validation logic
<code>validate_price()</code>	Accepts only positive numbers	Validates without crashing
<code>validate_category()</code>	Accepts only letter-based input	Ensures consistent data
<code>display_cars(cars)</code>	Prints all records in a table	Single reusable display routine
<code>add_car(cars)</code>	Adds a validated new record	Independent add operation
<code>update_car(cars)</code>	Edits an existing record	Independent update operation
<code>delete_car(cars)</code>	Removes a record by ID	Independent delete operation
<code>search_car(cars)</code>	Finds a record by ID	Encapsulates the linear search
<code>sort_by_price_asc(cars)</code>	Sorts by price, lowest first	Reuses the display routine
<code>sort_by_id_desc(cars)</code>	Sorts by ID, highest first	Reuses the display routine
<code>generate_report(cars)</code>	Computes inventory statistics	Report generation feature
<code>main()</code>	Runs the menu loop	Coordinates all modules

3.4 Algorithmic Logic Discussion

The program is controlled by a main menu loop implemented with a while statement, which keeps displaying the menu until the user chooses to exit. Inside the loop, an if/elif/else decision structure routes each numeric choice to the relevant function, and any unrecognised option prints an error message instead of crashing. Input validation is a key part of the logic. Car IDs are validated with a regular expression (r'[A-Za-z]+\d+') that requires letters followed by digits, and a linear scan rejects duplicates. Price input is wrapped in a try/except block around a float conversion, so non-numeric values are caught and the user is re-prompted, while a comparison rejects negative or zero prices. Category input is checked to contain only letters and spaces. Searching uses a linear search that compares each car's ID until a match is found. Sorting uses the built-in sorted() function with a lambda key that extracts price or ID, and descending ID order is achieved with reverse=True. Because the inventory is a list of dictionaries, update and delete modify the shared structure, after which save_inventory() persists the changes. These algorithms were chosen for their simplicity and suitability to the small scale of the inventory (Python Software Foundation, 2024).

4. Python Implementation

The ICMMS is implemented in a single Python file, icmms.py, containing fourteen functions organised around a central main() routine. The program uses only the standard library (json, re and pathlib), so it runs on any Python 3 installation without extra dependencies. The code follows PEP 8 (van Rossum, 2001): meaningful names, four-space indentation, blank lines between functions, and docstrings and comments explaining each function. Persistence is handled by load_inventory() and save_inventory(), which read from and write to the car_inventory.json text file. Records are saved after every add, update and delete operation, and again on the safe exit option, guaranteeing that no data is lost. The interface is menu-driven with clear, aligned tabular output that is easy for non-technical staff to use. The full source code is reproduced in Appendix B, and the program passed all of the tests described in Section 6.

5. Evidence of Program Run

To provide evidence that the functions work correctly, the program was run using a scripted sequence of inputs covering every menu option, including valid and invalid data and edge cases. Figure 1 shows an excerpt of the run illustrating the main menu and the price-ascending sort. The complete console log is reproduced in Appendix A. The run confirms that the program displays the inventory, adds cars, rejects invalid IDs, negative prices, invalid categories and duplicate IDs, searches for existing and missing cars, sorts by price and by ID, updates and deletes records, generates the summary report and exits safely after saving all records. No errors or crashes occurred during execution.

Figure 1: Excerpt of the program run

```
aaha@aaha-IdeaPad-Slim-3-15IAH8:/media/aaha/AAHA1/Unknown$ python3 icmms.py

===== KITT ICMS MAIN MENU =====
1. Display all cars
2. Add a new car
3. Update a car record
4. Delete a car record
5. Search a car by ID
6. Sort cars by price (ascending)
7. Sort cars by ID (descending)
8. Generate inventory summary report
9. Save and Exit
=====
Choose an option: 1
-----
ID          Name          Category    Price
-----
C101      AutoDrive X      Autonomous   75000.00
C102      SmartAI Pro      AI-Enabled   68000.00
C103      RoboCar Z        Electric     82000.00
```

6. Testing, Debugging and Documentation

Testing used black-box and edge-case techniques: the program was executed with known input data and the actual output was compared with the expected output. Table 3 presents the test plan. All fifteen test cases passed, confirming that every functional requirement is met.

Table 3: Test Plan with Expected and Actual Outputs

Test ID	Input Data	Expected Output	Actual Output	Result
T01	Option 1 (display)	Table with 3 cars	Table with 3 cars	Pass
T02	2, C104, EcoCar S, Electric, 45000	“Car added successfully”	As expected	Pass
T03	2, 123 (invalid ID)	“Invalid Car ID”	As expected	Pass

		message		
T04	2, C101 (duplicate)	“Car ID already exists”	As expected	Pass
T05	2, C105, Truck X, Autonomous, -5	Reject, then re-prompt	As expected	Pass
T06	2, C105, Truck X, Autonomous, abc	Reject, then re-prompt	As expected	Pass
T07	2, C106, MiniCar, Electri123	Reject, then re-prompt	As expected	Pass
T08	5, C101 (search)	Car C101 details	As expected	Pass
T09	5, C999 (search)	“Car not found”	As expected	Pass
T10	6 (sort by price)	Ascending price order	As expected	Pass
T11	7 (sort by ID)	Descending ID order	As expected	Pass
T12	3, C105, Truck XL, keep, y, 52000	“Car record updated”	As expected	Pass
T13	4, C106 (delete)	“Car record deleted”	As expected	Pass
T14	8 (report)	Summary with totals	As expected	Pass
T15	9 (exit)	All records saved, exit	As expected	Pass

6.1 Debugging

During development, several errors were corrected. First, a `ValueError` crash occurred when a user entered non-numeric text for the price, because the value was converted directly to a float. This was fixed by wrapping the conversion in a `try/except` block inside `validate_price()`, which now re-prompts the user. Second, the program accepted duplicate IDs, so a uniqueness check was added to `validate_car_id()`. Third, IDs were accepted in any format, so a regular expression was added to enforce the letter-then-digit pattern. Fourth, data was lost if the user closed the program without saving; records are now saved after every add, update and delete and on exit. Finally, the table output was misaligned because of inconsistent column widths, which was corrected in `display_cars()`. These fixes demonstrate systematic debugging using tracing and exception handling (Guttag, 2013).

7. Limitations

The current ICMMS is a terminal-based, single-user application. It stores data in a flat JSON text file, which suits a small inventory but is harder to manage at large scale. It has no database, so concurrent access is unsupported. Searching works only by car ID, not by name, category or price range. Categories are not restricted to a fixed list, so

spelling variations could affect consistency. There is no graphical interface, barcode scanning or customer module. These limitations do not prevent the system from meeting the current requirements but show where the design could be extended.

8. Future Improvements

The system could be extended in several ways. A graphical interface using Tkinter would make it more accessible. Replacing the JSON file with a database such as SQLite would improve scalability, concurrency and data integrity. Barcode scanning could identify cars automatically on the production line. A customer inquiry module could let customers view models and prices. Advanced reporting, such as exporting to Excel or CSV and charting price trends, would support decision-making, and staff authentication would prevent unauthorised changes. These enhancements align with the requirement that the system be reusable and scalable, so that new features can be added without rewriting existing modules (Downey, 2015).

9. Reflection

Developing the ICMMS was a valuable learning experience. The most important lesson was the value of modular design: breaking the program into small functions made it easier to write, test and debug. I also learned that validation must handle the mistakes users are most likely to make, such as typing letters instead of numbers or entering duplicate IDs, and that clear error messages matter as much as the validation itself. Testing taught me to plan cases using both valid and invalid data and to compare expected with actual output instead of assuming the program works. One alignment error in the table output was only noticed during testing, which reinforced the value of a structured test plan. Overall, this assessment deepened my understanding of Python programming, algorithmic thinking and the discipline needed to deliver reliable software.

10. Conclusion

This report presented the design, implementation, testing and evaluation of the Intelligent Car Manufacturing Management System for KITT Industries. The menu-driven application addresses the problems in the brief by allowing staff to add, display, update, delete, search and sort car records, with full input validation and

persistent storage to prevent data loss. The modular design follows PEP 8, and the test plan confirms that every feature works as expected. Although the system has limitations, they are consistent with the assignment scope and can be addressed by the proposed improvements. Overall, the ICMMS shows that a simple, well-structured program can significantly improve the accuracy and efficiency of inventory management.

References

- Anglia Ruskin University (2013) Guide to Harvard referencing. Cambridge: Anglia Ruskin University. Available at:
<https://library.aru.ac.uk/referencing/harvard.htm> (Accessed: 12 August 2026).
- Downey, A.B. (2015) Think Python: How to think like a computer scientist. 2nd edn. Sebastopol, CA: O'Reilly Media.
- Guttag, J.V. (2013) Introduction to computation and programming using Python. Cambridge, MA: MIT Press.
- Python Software Foundation (2024) The Python Tutorial. Available at:
<https://docs.python.org/3/tutorial/> (Accessed: 12 August 2026).
- van Rossum, G. (2001) PEP 8 - Style guide for Python code. Available at:
<https://peps.python.org/pep-0008/> (Accessed: 12 August 2026).
- Zelle, J.M. (2017) Python programming: An introduction to computer science. 3rd edn. Portland, OR: Franklin, Beedle and Associates.
- All in-text citations above follow the Harvard referencing system (Anglia Ruskin University, 2013).*

Appendix A - Full Console Log of Program Run

Figure 1: Excerpt of the program run with default value

```
aaha@aaha-IdeaPad-Slim-3-15IAH8:/media/aaha/AAHA1/Unknown$ python3 icmms.py

===== KITT ICMMS MAIN MENU =====
1. Display all cars
2. Add a new car
3. Update a car record
4. Delete a car record
5. Search a car by ID
6. Sort cars by price (ascending)
7. Sort cars by ID (descending)
8. Generate inventory summary report
9. Save and Exit
=====
Choose an option: 1
-----
ID          Name          Category    Price
-----
C101      AutoDrive X      Autonomous  75000.00
C102      SmartAI Pro      AI-Enabled  68000.00
C103      RoboCar Z        Electric    82000.00
```

Figure 2: Create With Read

```
===== KITT ICMMS MAIN MENU =====
1. Display all cars
2. Add a new car
3. Update a car record
4. Delete a car record
5. Search a car by ID
6. Sort cars by price (ascending)
7. Sort cars by ID (descending)
8. Generate inventory summary report
9. Save and Exit
=====
Choose an option: 2
Enter Car ID: C104
Enter Car Name: Tesla
Enter Category (e.g. Autonomous, AI-Enabled, Electric): Electric
Enter Price: 65000
Car added successfully.

===== KITT ICMMS MAIN MENU =====
1. Display all cars
2. Add a new car
3. Update a car record
4. Delete a car record
5. Search a car by ID
6. Sort cars by price (ascending)
7. Sort cars by ID (descending)
8. Generate inventory summary report
9. Save and Exit
=====
Choose an option: 1
-----
ID          Name          Category    Price
-----
C101      AutoDrive X      Autonomous  75000.00
C102      SmartAI Pro      AI-Enabled  68000.00
C103      RoboCar Z        Electric    82000.00
C104      Tesla            Electric    65000.00
```

Figure 3: Update with Read

```
===== KITT ICMMS MAIN MENU =====
1. Display all cars
2. Add a new car
3. Update a car record
4. Delete a car record
5. Search a car by ID
6. Sort cars by price (ascending)
7. Sort cars by ID (descending)
8. Generate inventory summary report
9. Save and Exit
=====
Choose an option: 3
Enter Car ID to update: C104
Current name: Tesla. New name (press Enter to keep): BYD
Current category: Electric. New category (press Enter to keep): Electric
Current price: 65000.0. Update price? (y/n): n
Car record updated successfully.

===== KITT ICMMS MAIN MENU =====
1. Display all cars
2. Add a new car
3. Update a car record
4. Delete a car record
5. Search a car by ID
6. Sort cars by price (ascending)
7. Sort cars by ID (descending)
8. Generate inventory summary report
9. Save and Exit
=====
Choose an option: 1
-----
ID          Name          Category      Price
-----
C101      AutoDrive X      Autonomous    75000.00
C102      SmartAI Pro      AI-Enabled    68000.00
C103      RoboCar Z        Electric      82000.00
C104      BYD              Electric      65000.00
```

Figure 4: Delete with Read

```
===== KITT ICMMS MAIN MENU =====
1. Display all cars
2. Add a new car
3. Update a car record
4. Delete a car record
5. Search a car by ID
6. Sort cars by price (ascending)
7. Sort cars by ID (descending)
8. Generate inventory summary report
9. Save and Exit
=====
Choose an option: 4
Enter Car ID to delete: C104
Car record deleted successfully.

===== KITT ICMMS MAIN MENU =====
1. Display all cars
2. Add a new car
3. Update a car record
4. Delete a car record
5. Search a car by ID
6. Sort cars by price (ascending)
7. Sort cars by ID (descending)
8. Generate inventory summary report
9. Save and Exit
=====
Choose an option: 1
-----
ID          Name          Category      Price
-----
C101      AutoDrive X      Autonomous    75000.00
C102      SmartAI Pro      AI-Enabled    68000.00
C103      RoboCar Z        Electric      82000.00
```

Figure 5: Read by value

```
===== KITT ICMMS MAIN MENU =====
1. Display all cars
2. Add a new car
3. Update a car record
4. Delete a car record
5. Search a car by ID
6. Sort cars by price (ascending)
7. Sort cars by ID (descending)
8. Generate inventory summary report
9. Save and Exit
=====
Choose an option: 5
Enter Car ID to search: C102
ID: C102, Name: SmartAI Pro, Category: AI-Enabled, Price: 68000
```

Figure 6: Shorting

```
===== KITT ICMMS MAIN MENU =====
1. Display all cars
2. Add a new car
3. Update a car record
4. Delete a car record
5. Search a car by ID
6. Sort cars by price (ascending)
7. Sort cars by ID (descending)
8. Generate inventory summary report
9. Save and Exit
=====
Choose an option: 6
-----
ID          Name          Category      Price
-----
C102        SmartAI Pro    AI-Enabled    68000.00
C101        AutoDrive X   Autonomous    75000.00
C103        RoboCar Z     Electric      82000.00
```

Figure 7: Shorting

```
===== KITT ICMMS MAIN MENU =====
1. Display all cars
2. Add a new car
3. Update a car record
4. Delete a car record
5. Search a car by ID
6. Sort cars by price (ascending)
7. Sort cars by ID (descending)
8. Generate inventory summary report
9. Save and Exit
=====
Choose an option: 7
-----
ID          Name          Category      Price
-----
C103        RoboCar Z     Electric      82000.00
C102        SmartAI Pro    AI-Enabled    68000.00
C101        AutoDrive X   Autonomous    75000.00
```

Figure 8: Summary

```
===== KITT ICMMS MAIN MENU =====
1. Display all cars
2. Add a new car
3. Update a car record
4. Delete a car record
5. Search a car by ID
6. Sort cars by price (ascending)
7. Sort cars by ID (descending)
8. Generate inventory summary report
9. Save and Exit
=====
Choose an option: 8

===== INVENTORY SUMMARY REPORT =====
Total number of cars:      3
Total inventory value:    $225,000.00
Average price:            $75,000.00
Cheapest car:             SmartAI Pro ($68,000.00)
Most expensive car:       RoboCar Z ($82,000.00)
```

Figure 9: Exit

```
===== KITT ICMMS MAIN MENU =====
1. Display all cars
2. Add a new car
3. Update a car record
4. Delete a car record
5. Search a car by ID
6. Sort cars by price (ascending)
7. Sort cars by ID (descending)
8. Generate inventory summary report
9. Save and Exit
=====
Choose an option: 9
All records saved. Thank you for using KITT ICMMS. Goodbye!
o aaha@aaha-IdeaPad-Slim-3-15IAH8:/media/aaha/AAHA1/Unknown$
```

Appendix B - Python Source Code (icmms.py)

Full source code of the ICMMS program.

```
"""
KITTT Industries - Intelligent Car Manufacturing Management System (ICMMS)

A menu-driven Python application that allows staff to manage car records,
organise vehicle data, perform sorting operations, and quickly search for
specific cars. All records are stored in a simple JSON text file so that
no data is lost when the program exits.

Author: KITTT Industries
"""

import json
import re
from pathlib import Path
```

```

FILE_NAME = "car_inventory.json"

def load_inventory():
    """Load car records from the JSON storage file."""
    path = Path(FILE_NAME)
    if path.exists():
        with open(path, "r", encoding="utf-8") as file:
            return json.load(file)
    return []

def save_inventory(cars):
    """Save all car records to the JSON storage file."""
    with open(FILE_NAME, "w", encoding="utf-8") as file:
        json.dump(cars, file, indent=4)

def validate_car_id(cars, car_id):
    """Return True if the Car ID has a valid format and is not a
    duplicate."""
    # A valid ID must start with letters and end with digits (e.g. C101).
    if not re.fullmatch(r"[A-Za-z]+\d+", car_id):
        print("Invalid Car ID. Use letters followed by digits, e.g. C101.")
        return False
    if any(car["id"].upper() == car_id.upper() for car in cars):
        print("Car ID already exists. Please use a different ID.")
        return False
    return True

def validate_price():
    """Prompt for a price and return it only if it is a positive number."""
    while True:
        try:
            price = float(input("Enter Price: "))
            if price <= 0:
                print("Price must be a positive number.")
                continue
            return price
        except ValueError:
            print("Invalid price. Please enter a number.")

def validate_category():
    """Prompt for a category and return it only if it is valid text."""
    while True:
        category = input("Enter Category (e.g. Autonomous, AI-Enabled,
Electric): ").strip()
        if category and category.replace(" ", "").isalpha():
            return category
        print("Invalid category. Use letters only (no numbers or symbols).")

def display_cars(cars):
    """Display every car record in a readable table."""
    if not cars:
        print("No cars available.")
        return

    print("-" * 64)
    print(f"{'ID':<10}{'Name':<20}{'Category':<15}{'Price':>10}")
    print("-" * 64)

```



```

        for car in cars:

print(f"{car['id']:<10}{car['name']:<20}{car['category']:<15}{car['price']:>10.2f}")

def add_car(cars):
    """Add a new car record after validating all input values."""
    car_id = input("Enter Car ID: ").strip().upper()

    if not validate_car_id(cars, car_id):
        return

    name = input("Enter Car Name: ").strip()
    if not name:
        print("Car name cannot be empty.")
        return

    category = validate_category()
    price = validate_price()

    cars.append({
        "id": car_id,
        "name": name,
        "category": category,
        "price": price
    })

    print("Car added successfully.")

def update_car(cars):
    """Update the name, category, and price of an existing car record."""
    car_id = input("Enter Car ID to update: ").strip().upper()

    for car in cars:
        if car["id"] == car_id:
            new_name = input(f"Current name: {car['name']}. New name (press Enter to keep): ").strip()
            if new_name:
                car["name"] = new_name

            new_category = input(
                f"Current category: {car['category']}. New category (press Enter to keep): ").strip()
            if new_category and new_category.replace(" ", "").isalpha():
                car["category"] = new_category
            elif new_category:
                print("Invalid category. Keeping the current value.")

            update_price = input(f"Current price: {car['price']}. Update price? (y/n): ").strip().lower()
            if update_price == "y":
                car["price"] = validate_price()

            print("Car record updated successfully.")
            return

    print("Car not found.")

```



```

def delete_car(cars):
    """Remove an existing car record by its Car ID."""
    car_id = input("Enter Car ID to delete: ").strip().upper()

    for car in cars:
        if car["id"] == car_id:
            cars.remove(car)
            print("Car record deleted successfully.")
            return

    print("Car not found.")

def search_car(cars):
    """Search for a single car record by its Car ID."""
    car_id = input("Enter Car ID to search: ").strip().upper()

    for car in cars:
        if car["id"] == car_id:
            print(f"ID: {car['id']}, Name: {car['name']}, "
                  f"Category: {car['category']}, Price: {car['price']}")
            return

    print("Car not found.")

def sort_by_price_asc(cars):
    """Display all cars sorted by price in ascending order."""
    display_cars(sorted(cars, key=lambda car: car["price"]))

def sort_by_id_desc(cars):
    """Display all cars sorted by Car ID in descending order."""
    display_cars(sorted(cars, key=lambda car: car["id"], reverse=True))

def generate_report(cars):
    """Print a summary report with inventory statistics."""
    if not cars:
        print("No data available to generate a report.")
        return

    total_cars = len(cars)
    total_value = sum(car["price"] for car in cars)
    average_price = total_value / total_cars
    cheapest = min(cars, key=lambda car: car["price"])
    most_expensive = max(cars, key=lambda car: car["price"])

    print("\n===== INVENTORY SUMMARY REPORT =====")
    print(f"Total number of cars:           {total_cars}")
    print(f"Total inventory value:             ${total_value:,.2f}")
    print(f"Average price:                     ${average_price:,.2f}")
    print(f"Cheapest car:                      {cheapest['name']}")
    print(f"({cheapest['price']:,.2f})")
    print(f"Most expensive car:                 {most_expensive['name']}")
    print(f"({most_expensive['price']:,.2f})")
    print("=====")

def main():
    """Run the main menu loop of the ICMMS application."""
    cars = load_inventory()

```

```

while True:
    print("\n===== KITT ICMMS MAIN MENU =====")
    print("1. Display all cars")
    print("2. Add a new car")
    print("3. Update a car record")
    print("4. Delete a car record")
    print("5. Search a car by ID")
    print("6. Sort cars by price (ascending)")
    print("7. Sort cars by ID (descending)")
    print("8. Generate inventory summary report")
    print("9. Save and Exit")
    print("=====")

    choice = input("Choose an option: ").strip()

    if choice == "1":
        display_cars(cars)
    elif choice == "2":
        add_car(cars)
        save_inventory(cars) # Save immediately to prevent data loss.
    elif choice == "3":
        update_car(cars)
        save_inventory(cars)
    elif choice == "4":
        delete_car(cars)
        save_inventory(cars)
    elif choice == "5":
        search_car(cars)
    elif choice == "6":
        sort_by_price_asc(cars)
    elif choice == "7":
        sort_by_id_desc(cars)
    elif choice == "8":
        generate_report(cars)
    elif choice == "9":
        save_inventory(cars)
        print("All records saved. Thank you for using KITT ICMMS.")
        Goodbye!")
        break
    else:
        print("Invalid option. Please choose a number from 1 to 9.")

if __name__ == "__main__":
    main()

```